Linear Regression

1. Problem Statement

The process of creating and evaluating a simple linear regression model using Fish.csv

2. Load the dataset

```
import numpy as np
import pandas as pd
df=pd.read_csv("Fish.csv")
```

3. select the most appropriate feature(s) from Fish.csv for your single linear regression model to predict the weight of fish?

```
df.corr(numeric_only=True)
```

4. Data Splitting

take a test_size=0.2 and random_state=3

```
from sklearn.model_selection import train_test_split
X = df[['Length3']]
y = df['Weight']
```

or (for multilinear regression)

```
X = df[["Length1", "Length2", "Length3", "Height", "Width"]]
y = df['Weight']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=3)
```

5. Model Fitting

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

6. Prediction

```
y_pred = model.predict(X_test)
```

7. R2 Score and MSE

```
from sklearn.metrics import r2_score, mean_squared_error
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
```

9. Predict the weight value if Length3=30

```
pred=model.predict(np.array([30]).reshape(1,-1))
```

or (for multilinear regression)

Predict the weight value if Length1=20,Length2=20,Lenghth3=30, height=12, width=5

```
pred=model.predict(np.array([[20,20,30,12,5]]))
```

polynomial regression model

1. Problem defination

Creating a polynomial regression model using Fish.csv involves several steps, from feature selection to evaluating the model's performance.

2. Load the Dataset¶

```
fish_data=pd.read_csv("Fish.csv")
```

3. take a features(iindependent): Length1, Length2, Length3, Height, Width
target(dependent): Weight

```
X = fish_data[['Length1', 'Length2', 'Length3', 'Height', 'Width']]
y = fish_data['Weight']
```

4. Data Splitting¶

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=25)
```

5. Model Fitting

- Transform the Features: Use PolynomialFeatures from sklearn.preprocessing to transform the original features into polynomial features.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
# Transform features to polynomial features
degree = 3  # Example degree
poly = PolynomialFeatures(degree=degree)
X_poly_train = poly.fit_transform(X_train)
X_poly_test = poly.transform(X_test)
# Fit the model
model = LinearRegression()
model.fit(X_poly_train, y_train)
```

6. Prediction

```
y_pred = model.predict(X_test)
```

7. R2 Score and MSE

```
from sklearn.metrics import r2_score, mean_squared_error
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')
```

9 Predict the weight value if Length1=20,Length2=20,Lenghth3=30, height=12, width=5

```
pred=model.predict(poly.transform([[20,20,30,12,5]]))
```

In k-Nearest Neighbors (k-NN) classification and decision tree , the target column (i.e., the labels) does not necessarily need to be converted to numerical values

KNN Model

1. Problem Defination

Creating a KNN (K-Nearest Neighbors) model for Customer-Churn.csv involves several steps, from selecting relevant features to evaluating and optimizing the model

2. Load Dataset¶

```
churn_data= pd.read_csv('Churn.csv')
```

3. fill the missing data

```
churn_data.TotalCharges=churn_data.TotalCharges.fillna(churn_data.TotalCharges.mean())
```

one hot encoding

Encode the categorical variables

```
categorical_features = ['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines',
'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',  'TechSupport',
'StreamingTV', 'StreamingMovies', 'Contract',  'PaperlessBilling', 'PaymentMethod']
# One-hot encode categorical variables
churn_data = pd.get_dummies(churn_data, columns=categorical_features, drop_first=True)
```

4. Select features and target and Select features and target

```
X = churn_data.drop(['customerID', 'Churn',"Unnamed: 0"], axis=1)
y = churn_data['Churn']
```

4. Data Splitting

- What steps are involved in splitting Fish.csv into training (80%) and testing (20%) sets using a random state of 0.2?

```
from sklearn.model_selection import train_test_split
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
```

5. Model Training and fit

- How do you train a KNN model using the training set from Fish.csv?

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
# Fit the model
knn.fit(X_train, y_train)
```

6.Prediction

- How do you use a trained KNN model to predict the species of fish in the test set from Fish.csv?

```
y_pred = knn.predict(X_test)
```

7 Model Evaluation - Accuracy,recall,precision,classification report

What methods do you use to measure the accuracy of your KNN model on Churn.csv?

```
from sklearn.metrics import confusion_matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(conf_matrix)
# True Positives (TP), True Negatives (TN), False Positives (FP), False Negatives (FN)
TP = conf_matrix[1, 1]
TN = conf_matrix[0, 0]
FP = conf_matrix[0, 1]
FN = conf_matrix[1, 0]
print(TP,TN,FP,FN)
# Calculate Accuracy
accuracy = (TP + TN) / (TP + TN + FP + FN)
# Calculate Recall (Sensitivity)
recall = TP / (TP + FN)
# Calculate Precision
```

```
precision = TP / (TP + FP)
# Calculate Specificity
specificity = TN / (TN + FP)
print(f'Accuracy: {accuracy}')
print(f'Recall (Sensitivity): {recall}')
print(f'Precision: {precision}')
print(f'Specificity: {specificity}')
from sklearn.metrics import
accuracy_score,precision_score,recall_score,f1_score,classification_report
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')
# Recall
recall = recall_score(y_test, y_pred,pos_label="Yes")
print(f'Recall: {recall}')
# Precision
precision = precision_score(y_test, y_pred,pos_label="Yes")
print(f'Precision: {precision}')
# F1 Score
f1 = f1_score(y_test, y_pred,pos_label="Yes")
print(f'F1 Score: {f1}')
# Classification Report
class_report = classification_report(y_test, y_pred)
print('Classification Report:')
print(class_report)
```

Decision Tree Classifier

1. Problem Defination

- To create a decision tree classifier model using the provided dataset Churn.csv

2. Load the Dataset

```
churn_data=pd.read_csv("Churn.csv")
```

3. Feature Selection

```
churn_data.TotalCharges=churn_data.TotalCharges.fillna(churn_data.TotalCharges.mean())
categorical_features = ['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines',
'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',  'TechSupport',
'StreamingTV', 'StreamingMovies', 'Contract',  'PaperlessBilling', 'PaymentMethod']
# One-hot encode categorical variables
churn_data = pd.get_dummies(churn_data, columns=categorical_features, drop_first=True)
```

4. Split the data

- test_size=0.2 and random_state=2

```
from sklearn.model_selection import train_test_split
# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)
```

5. Train and Fit the Model

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
# Train the Decision Tree Classifier
dt_classifier = DecisionTreeClassifier(criterion="entropy",random_state=42,max_depth=2)
dt_classifier.fit(X_train, y_train)
```

6. Predict the model

```
# Make predictions
y_pred = dt_classifier.predict(X_test)
from sklearn import tree
text_representation = tree.export_text(dt_classifier)
print(text_representation)
```

7. Evalute the model

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
# True Positives (TP), True Negatives (TN), False Positives (FP), False Negatives (FN)
TP = conf_matrix[1, 1]
TN = conf_matrix[0, 0]
FP = conf_matrix[0, 1]
FN = conf_matrix[1, 0]
print(TP,TN,FP,FN)
# Calculate Accuracy
accuracy = (TP + TN) / (TP + TN + FP + FN)
# Calculate Recall (Sensitivity)
recall = TP / (TP + FN)
# Calculate Precision
precision = TP / (TP + FP)
# Calculate Specificity
specificity = TN / (TN + FP)
print(f'Accuracy: {accuracy}')
print(f'Recall (Sensitivity): {recall}')
print(f'Precision: {precision}')
```

print(f'Specificity: {specificity}')

Adding a feature called area from length and breadth

data['Area'] = data['Length'] * data['Breadth']

price convert into category

data4['Price'] = np.where(data4['Price'] > 3000000, 'High', np.where(data4['Price'] < 2000000, 'Low', 'Medium'))

Random Forest Classifier

1. Problem Definition

• To create a Random Forest Classifier model using the provided dataset Churn.csv

2. Load the Dataset

```
import pandas as pd
churn_data = pd.read_csv("Churn.csv")
```

3. Feature Selection & Preprocessing

```
# Handle missing values
churn_data.TotalCharges = churn_data.TotalCharges.fillna(churn_data.TotalCharges.mean())
# Categorical features
categorical_features = ['gender', 'Partner', 'Dependents', 'PhoneService',  'MultipleLines',
'InternetService', 'OnlineSecurity',  'OnlineBackup', 'DeviceProtection', 'TechSupport',
'StreamingTV', 'StreamingMovies', 'Contract',  'PaperlessBilling', 'PaymentMethod']
# One-hot encoding
churn_data = pd.get_dummies(churn_data, columns=categorical_features, drop_first=True)
# Define X and y
X = churn_data.drop("Churn", axis=1)
y = churn_data["Churn"]
```

4. Split the Data

• test_size = 0.2 and random_state = 2

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=2)
```

5. Train and Fit the Model

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
# Train Random Forest
rf = RandomForestClassifier( n_estimators=100,  max_depth=4, random_state=42)
rf.fit(X_train, y_train)
```

6. Predict the Model

```
# Predictions
y_pred = rf.predict(X_test)
```

7. Evaluate the Model

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
```

```
# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
TP = conf_matrix[1, 1]
TN = conf_matrix[0, 0]
FP = conf_matrix[0, 1]
FN = conf_matrix[1, 0]
print(TP, TN, FP, FN)
# Manual Calculations
accuracy = (TP + TN) / (TP + TN + FP + FN)
recall = TP / (TP + FN)
precision = TP / (TP + FP)
specificity = TN / (TN + FP)
print(f'Accuracy: {accuracy}')
print(f'Recall (Sensitivity): {recall}')
print(f'Precision: {precision}')
print(f'Specificity: {specificity}')
```

Support Vector Machine Classifier

1. Problem Definition

• To create a Random Forest Classifier model using the provided dataset Churn.csv

2. Load the Dataset

```
import pandas as pd
churn_data = pd.read_csv("Churn.csv")
```

3. Feature Selection & Preprocessing

```
# Handle missing values
churn_data.TotalCharges = churn_data.TotalCharges.fillna(churn_data.TotalCharges.mean())
# Categorical features
categorical_features = ['gender', 'Partner', 'Dependents', 'PhoneService',  'MultipleLines',
'InternetService', 'OnlineSecurity',  'OnlineBackup', 'DeviceProtection', 'TechSupport',
'StreamingTV', 'StreamingMovies', 'Contract',  'PaperlessBilling', 'PaymentMethod']
# One-hot encoding
churn_data = pd.get_dummies(churn_data, columns=categorical_features, drop_first=True)
# Define X and y
X = churn_data.drop("Churn", axis=1)
y = churn_data["Churn"]
```

4. Split the Data

• test_size = 0.2 and random_state = 2

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=2)
```

5. Train and Fit the Model

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
# Train Random Forest
svm = SVC(C=10,kernel='rbf', random_state=42)
svm.fit(X_train, y_train)
```

6. Predict the Model

```
# Predictions
y_pred = svm.predict(X_test)
```

7. Evaluate the Model

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
TP = conf_matrix[1, 1]
TN = conf_matrix[0, 0]
FP = conf_matrix[0, 1]
FN = conf_matrix[1, 0]
print(TP, TN, FP, FN)
# Manual Calculations
accuracy = (TP + TN) / (TP + TN + FP + FN)
recall = TP / (TP + FN)
precision = TP / (TP + FP)
specificity = TN / (TN + FP)
print(f'Accuracy: {accuracy}')
print(f'Recall (Sensitivity): {recall}')
print(f'Precision: {precision}')
print(f'Specificity: {specificity}')
```

The mathematical formulas for the confusion matrix and the evaluation metrics, using the terms True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN):

Confusion Matrix:

For a binary classification problem (like cat vs. dog),

	Predicted Negative	**Predicted Positive**
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

Evaluation Metrics:

1. Accuracy:

$$Accuracy = \frac{(TP + TN)}{(FP + FN + TP + TN)}$$

2. Precision (for a specific class, e.g., 'positive'):

$$Precision = \frac{(TP)}{(FP + TP)}$$

3. Sensitivity (Recall, True Positive Rate) (for a specific class, e.g., 'positive'):

$$Sensitivity = Recall = \frac{(TP)}{(FN + TP)}$$

4. Specificity (True Negative Rate) (for a specific class, e.g., 'positive', where 'negative' is the other class):

$$Specificity = \frac{(TN)}{(FP + TN)}$$

5. F1-Score (for a specific class, e.g., 'positive'):

$$F1 = \frac{2 \times (PrecisionXRecall)}{(Precision + Recall)}$$

Mathematical Formulas

For a binary classification problem (like cat vs. dog), the confusion matrix looks like this:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

1. Confusion Matrix:

As shown in the table above, it's a table that summarizes the performance of a classification model by showing the counts of true positives, true negatives, false positives, and false negatives.

2. Accuracy:

$$Accuracy = \frac{Number\ of\ Correct\ Predictions}{Total\ Number\ of\ Predictions} = \frac{TP + TN}{FP + FN + TP + TN}$$

3. Precision:

Precision measures the proportion of correctly predicted positive instances out of all instances predicted as positive. For 'cat' as the positive class:

$$Precision_{cat} = \frac{TP_{cat}}{FP_{cat} + TP_{cat}}$$

For 'dog' as the positive class:

$$Precision_{dog} = \frac{TP_{dog}}{FP_{dog} + TP_{dog}}$$

4. Sensitivity (Recall):

Sensitivity (also known as recall or true positive rate) measures the proportion of actual positive instances that were correctly identified.

For 'cat' as the positive class:

$$Sensitivity_{cat} = Recall_{cat} = \frac{TP_{cat}}{FN_{cat} + TP_{cat}}$$

For 'dog' as the positive class:

$$Sensitivity_{dog} = Recall_{dog} = \frac{TP_{dog}}{FN_{dog} + TP_{dog}}$$

5. Specificity:

Specificity (also known as the true negative rate) measures the proportion of actual negative instances that were correctly identified.

For 'cat' as the positive class ('dog' is the negative class):

$$Specificity_{cat} = \frac{TN_{cat}}{FP_{cat} + TN_{cat}}$$

For 'dog' as the positive class ('cat' is the negative class):

$$Specificity_{dog} = \frac{TN_{dog}}{FP_{dog} + TN_{dog}}$$

6. F1-Score:

The F1-score is the harmonic mean of precision and recall. It provides a balanced measure of a model's performance.

$$For\ 'cat': F1_{cat} = 2 \times \frac{Precision_{cat} X Recall_{cat}}{Precision_{cat} + Recall_{cat}}$$

$$For\ 'cat': F1_{dog} = 2 \times \frac{Precision_{dog} X Recall_{dog}}{Precision_{dog} + Recall_{dog}}$$

- **Accuracy:** General correctness, but can be misleading on imbalanced data.
- **Precision:** How accurate are the positive predictions? (Avoids false positives)
- **Recall (Sensitivity):** How well does the model find all the actual positives? (Avoids false negatives)
- **Specificity:** How well does the model identify all the actual negatives? (Avoids false positives on the negative class's perspective)
- **F1-Score:** A balanced measure of precision and recall, useful when both false positives and false negatives are important.

Introduction to Error Metrics in Regression Analysis

In regression analysis, error metrics are used to evaluate the performance of a predictive model. They measure the difference between the predicted values and the actual values. The choice of error metric can significantly affect the interpretation of the model's accuracy and the subsequent decisions based on the model's predictions.

Here are some commonly used error metrics:

1. Mean Absolute Error (MAE)

Definition:
MAE measures the average magnitude of the errors in a set of predictions, without considering their direction. It is the average over the absolute differences between predicted and actual values.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

When to Use:

- When you need an interpretable metric in the same units as the response variable.
- When you want to minimize large outliers since MAE treats all errors equally.

Advantages:

- Easy to understand and interpret.
- Less sensitive to outliers compared to MSE.

Disadvantages:

- Might not be as informative for very large datasets due to ignoring the variance in error magnitudes.

2. Mean Squared Error (MSE)

Definition:
MSE measures the average of the squares of the errors. It is more sensitive to outliers than MAE because it squares the error terms.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

When to Use:

- When you want to heavily penalize larger errors.
- When normally distributed errors are assumed.

Advantages:

- The squaring process prevents canceling out of positive and negative errors.
- The metric is useful for gradient-based optimization algorithms.

Disadvantages:

- More sensitive to outliers, which can distort the performance measure if the dataset has many outliers.

3. Root Mean Squared Error (RMSE)

Definition:
RMSE is the square root of the mean squared error. It provides an error metric in the same units as the response variable.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

When to Use:

- When you need to interpret the error metric in the same units as the response variable.
- When comparing different models, especially if the dataset contains outliers.

Advantages:

- Like MSE, but with the benefit of being in the same units as the original data.
- Penalizes larger errors more heavily.

Disadvantages:

- Still sensitive to outliers like MSE.

4. R² Score (Coefficient of Determination)

Definition:
R² measures the proportion of the variance in the dependent variable that is predictable from the independent variables.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Where (\bar{y}) is the mean of the observed data.

When to Use:

- When you want to know how well your model explains the variability of the response data around its mean.

Advantages:

- Provides a relative measure of fit.
- Easy to interpret.

Disadvantages:

- Can give misleading results for non-linear models.

- Not suitable for comparing models with different numbers of predictors.

5. Adjusted R² Score

Definition:
Adjusted R² adjusts the R² value based on the number of predictors in the model, penalizing the addition of non-informative predictors.

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1-R^2)(n-1)}{n-k-1} \right)$$

Where (n) is the number of observations and (k) is the number of predictors.

When to Use:

- When comparing models with a different number of predictors.
- To avoid overfitting by penalizing additional predictors that do not add value.

Advantages:

- More accurate than R² for multiple regression models.
- Helps in model selection.

Disadvantages:

- Can still be misleading if the model includes irrelevant predictors.

Summary

Choosing the right error metric is crucial depending on the specific requirements and characteristics of your dataset. MAE, MSE, and RMSE offer different perspectives on model accuracy, with varying sensitivities to outliers. R² and Adjusted R² provide insights into the proportion of variance explained by the model, with Adjusted R² adjusting for the number of predictors. Understanding and correctly applying these metrics helps in building more robust and interpretable regression models.

In [1]:

```
### Code Examples for Error Metrics

##Here's how to compute these metrics using Python with `scikit-learn`:


import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2

# Example data
y_true = np.array([3.0, -0.5, 2.0, 7.0])
y_pred = np.array([2.5, 0.0, 2.1, 7.8])

# MAE
mae = mean_absolute_error(y_true, y_pred)
print(f'MAE: {mae}')

# MSE
mse = mean_squared_error(y_true, y_pred)
print(f'MSE: {mse}')

# RMSE
rmse = np.sqrt(mse)
print(f'RMSE: {rmse}')

# R² Score
r2 = r2_score(y_true, y_pred)
print(f'R²: {r2}')

# Adjusted R² Score (manual calculation)
n = len(y_true)
k = 1  # number of predictors, adjust based on your model
adjusted_r2 = 1 - ((1 - r2) * (n - 1) / (n - k - 1))
print(f'Adjusted R²: {adjusted_r2}')
```

```
MAE: 0.475
MSE: 0.2874999999999999
RMSE: 0.5361902647381803
R²: 0.9605995717344754
Adjusted R²: 0.9408993576017131
```

In [ ]:

```

```